

DEEP LEARNING

Labwork 4 - Neural Network

Student: Foo Cheung Nicolas - ES2540023

1 Algorithm Implementation

The objective of this labwork is to implement a neural network from scratch using object-oriented programming. The neural network supports feedforward propagation and can solve the XOR logical problem.

1.1 Network Architecture

The neural network is composed of several layers. Each layer contains multiple neurons, and each neuron contains:

- A list of weights
- A bias
- An activation function

The structure of the network is stored in a text file. Example:

```
3
2
2
1
```

This represents:

- 3 layers in total (input, hidden, output)
- 2 neurons in the input layer
- 2 neurons in the hidden layer
- 1 neuron in the output layer

1.2 Feedforward Implementation

For each neuron, the weighted sum is computed using:

$$z = \sum_{i=1}^n w_i x_i + b$$

where:

- w_i are the weights
- x_i are the inputs
- b is the bias

The activation output is then computed.

Initially, the sigmoid activation function was used:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

However, the XOR weights provided in the slides were designed for a perceptron using a step activation function. Therefore, the step function was used for the final XOR experiment:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

2 Object-Oriented Design

The network architecture is read from a text file. The implementation contains three main classes:

- Neuron
- Layer
- NeuralNetwork

2.1 Neuron Class

The `Neuron` class stores weights and bias, and computes the activation output.

```
class Neuron:
    def __init__(self, weights, bias):
        self.weights = weights
        self.bias = bias
```

2.2 Layer Class

The `Layer` class contains multiple neurons and computes outputs for the entire layer.

```
class Layer:
    def __init__(self, neurons):
        self.neurons = neurons
```

2.3 NeuralNetwork Class

The `NeuralNetwork` class contains all layers and performs feedforward propagation.

```
class NeuralNetwork:
    def __init__(self, layers):
        self.layers = layers
```

3 XOR Experiment

The XOR problem was tested using the following weights and biases.

3.1 Hidden Layer

Neuron 1 (NAND):

$$w = [-1, -1], \quad b = 1.5$$

Neuron 2 (OR):

$$w = [1, 1], \quad b = -0.5$$

3.2 Output Layer

Output neuron (AND):

$$w = [1, 1], \quad b = -1.5$$

4 Results

The network was tested using the four XOR inputs.

Input	Output
(0,0)	0
(0,1)	1
(1,0)	1
(1,1)	0

The neural network successfully reproduced the XOR logical function.